

BlackParrot: An Agile Open-Source RISC-V Multicore for Accelerator SoCs

Daniel Petrisko, Farzam Gilani, Mark Wyse, Dai Cheol Jung, Scott Davidson, Paul Gao, and Chun Zhao

University of Washington

Zahra Azad and Sadullah Canakci

Boston University

Bandhav Veluri and Tavio Guarino

University of Washington

Ajay Joshi

Boston University

Mark Oskin and Michael Bedford Taylor

University of Washington

Abstract—This article introduces BlackParrot, which aims to be the default open-source, Linux-capable, cache-coherent, 64-bit RISC-V multicore used by the world. In executing this goal, our research aims to advance the world’s knowledge about the “software engineering of hardware.” Although originally bootstrapped by the University of Washington and Boston University via DARPA funding, BlackParrot strives to be community driven and infrastructure agnostic; a multicore which is Pareto optimal in terms of power, performance, area, and complexity. In order to ensure BlackParrot is easy to use, extend, and, most importantly, trust, development is guided by three core principles: Be Tiny, Be Modular, and Be Friendly. Development efforts have prioritized the use of intentional interfaces and modularity and silicon validation as first-order design metrics, so that users can quickly get started and trust that their design will perform as

Digital Object Identifier 10.1109/MM.2020.2996145

Date of publication 20 May 2020; date of current version

30 June 2020.

expected when deployed. BlackParrot has been validated in a GlobalFoundries 12-nm FinFET tapeout. BlackParrot is ideal as a standalone Linux processor or as a malleable fabric for an agile accelerator SoC design flow.

■ **Risc-v⁹** is a disruptive technology. Never before has such a large, global community worked together to put forth a complete open source instruction set, machine model, and software stack. There is a strong belief in the community that RISC-V will find their foothold as low-NRE, high performance host cores to the agile-developed specialized accelerators that are being created to in response to the end of Dennard Scaling.

Strangely lacking, however, is a similarly globally maintained open source *implementation* of a RISC-V SoC. BlackParrot, described in this article, is designed to fill this gap. BlackParrot is open source and available now under the BSD-3 license. BlackParrot is written in standard SystemVerilog and hence effortlessly integrates into existing design methodologies and is easily understood and modified by industrial designers. BlackParrot has been fabricated in the GlobalFoundries 12-nm process, with several iterations of refactoring to attain high area, delay, and power efficiency.

Taking lessons from software engineering and the scalability of Linux development, BlackParrot has a modular design that puts its interfaces first. We believe that this will enable scalable collaboration by allowing contributors to work independently without having to completely understand all of BlackParrot's components, or how they are evolving. We seek not only to advance the state-of-the-art in open source processor architecture, but also to develop approaches that change the way the world designs hardware.

SUCCESS METRICS

We believe adoption of BlackParrot will be driven by optimizing across four dimensions, as shown in Figure 1: quality, virality, functionality, and efficiency.

Quality: From the beginning, BlackParrot was architected not simply to achieve RISC-V compliance, but to produce a quality codebase that engineers could inherently *trust* as a secure and high-quality design. At the heart of this approach (which we term informally *the software engineering of hardware*) is the pervasive use of intentionally designed, narrow, modular interfaces that make the design easy to reason about without sacrificing performance, power, or area. Leveraging years of processor design experience, we developed a high-level design document that partitioned the multicore into three major modules with easy-to-understand, lightweight transactional interfaces. Each module then had its own design document which specified its own internal modular interfaces, and worked through the important nuances and special cases. From there, we produced schematics and then SystemVerilog RTL which leveraged BaseJump Standard Template Library (STL),⁸ an expansive set of intentionally designed interfaces for common computer architecture and hardware atoms that comes with corresponding silicon-verified SystemVerilog implementations. The RTL then underwent extensive code review, and both unit and random testing. We are systematically measuring toggle, line, and functional coverage, and driving up the coverage of our verification methodology on a daily basis. The goal is that an experienced engineer can evaluate the documentation, code, and tests; appreciate BlackParrot's quality; and use it confidently.

Virality: While BlackParrot is a quality design, we realize it will ultimately be unsuccessful if it is not widely adopted and if the community does not collectively take stewardship of it. For this reason, we have focused on the out-of-the-box experience, making it simple to get up and

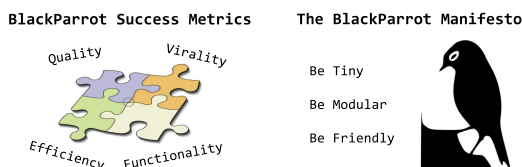


Figure 1. BlackParrot Success Metrics and Manifesto. The Success Metrics strategically align the project for widespread adoption, and the BlackParrot Manifesto provides tactical guidance for technical decisions.

	CoreMark/MHz (Higher better)	Language	Design Schema	Natively Multicore	License
BlackParrot	3.04	SystemVerilog	Pervasive Modular Interfaces; Standard Template Library	✓	BSD-3
Rocket	2.62	Chisel	Generator	✓	BSD-3
Ariane	2.45	SystemVerilog	Monolithic		SHL-2
Shakti	2.20	BlueSpec	Generator		BSD-3
Flute	1.00	BlueSpec	Generator		Apache-2
SiFive U54MC	3.01	Chisel	Generator	✓	Closed Source

Figure 2. Recent energy/performance optimized 64-bit RISC-V open source Linux-capable ASIC application class processor cores. Per-core performance is given in CoreMark/MHz as is industry standard. For reference, we give the equivalent closed source, for-money, Linux-capable SiFive U54 core. (Note that since these open source projects are living, breathing projects, this is just a snapshot in time!)

running via a github checkout, and pulling in as few external components as possible. We employ a widely known language, SystemVerilog, instead of Chisel or BlueSpec. We have a focus on friendliness and inclusiveness in our social interactions. Too many online collaboration forums are marred by a tolerance of abusive behavior particularly by respected members of the community, as highlighted by a recent case where Linus Torvalds himself stepped away from Linux for several months to try to contemplate the toxic effects of his curmudgeonly attitude. We actively try to prevent “not invented here” syndrome from taking hold in the BlackParrot effort. Our effort welcomes your contributions and the modular nature of the design makes it easy for individual contributors to get up to speed.

Functionality: BlackParrot boots Linux. It implements all of the core features of the RISC-V instruction set that are used by current software stacks. It also contains all the useful features required by modern SoCs, such as interrupt controllers, coherent cache hierarchies, and easy-to-integrate accelerator interfaces.

Efficiency: BlackParrot must have best-of-class PPA for its target domain; a Linux host multicore for accelerator chips. Early Coremark scores show BlackParrot achieves competitive performance with both academic and commercial cores of its class, as shown in Figure 2. Extensive design and RTL code review are used to ensure BlackParrot contains efficient implementations of modern microarchitectural features expected of a Linux-class microprocessor. SRAM and logic structures are sized to be

performance, power, and area (PPA) efficient. To validate the design we have fabricated BlackParrot in GlobalFoundries 12-nm process node (see Figure 4), and are deploying new features across frequent 12-nm and 40-nm tapeouts.

DESIGN MANIFESTO

During the course of the project, there have been many cases where there are two reasonable technical directions to take the project. To guide our effort, we developed an informal *manifesto* to help decide these difficult chases. The manifesto has the following three key rules.

- 1) *Be Tiny.* When choosing among alternatives, we choose the option that results in a smaller, more understandable code base and in less die area, simpler critical paths, and fewer bugs. The result is a code base that is as small and understandable as possible, and hardware that is PPA efficient. We take care to *not* implement esoteric and performance non-critical components in RTL, and to avoid a common problem in recent generator-based RTL methodologies: a multitude of tunable knobs in which most combinations have been untested and yield dubious PPA benefit. If a feature is required by the RISC-V spec but is not performance critical, we implement it through emulation. The code is “all the RTL you need and nothing that you do not.”
- 2) *Be Modular.* We employ clean, latency-insensitive interfaces that do not rely on knowledge of the other module’s internals. This allows multiple contributors to work

independently of each other, and to minimize bugs that emerge from incomplete understanding of the entire code base.

- 3) *Be Friendly*. We ask ourselves both in design decisions and in our presentation: Does this make the project more approachable? With this, we can build a large open source project culture that encourages contributions and avoids the “not invented here” syndrome.

System Architecture

The BlackParrot multicore implements the RISC-V 64-bit “RV64 G” architecture, which includes the base integer ISA “I,” multiplication and division “M,” atomics “A,” single and double precision floating-point “F/D.” It supports three privilege levels—machine, supervisor, and user—as well as SV39 virtual memory; these extensions are sufficient to efficiently run full-featured operating systems such as Linux.

Race-Free Programmable Cache Coherence

BlackParrot implements a distributed directory-based cache coherence protocol, which currently supports VI, MSI, and MESI. The underlying implementation, *BedRock*, consists of a collection of local cache engines (LCEs), each controlling an L1 cache, that connect over an interconnection network to the programmable cache coherence engines (CCEs), which collectively maintain the address-sharded directory state.

The BedRock protocol implementations have the unique property of being *race-free*, because they ensure that coherence state transitions occur at the CCEs and not at the local caches or LCEs, which significantly reduces protocol complexity.

Heterogeneously Tiled Multicore SoC Microarchitecture

BlackParrot is designed as a scalable, heterogeneously tiled multicore microarchitecture, as shown in Figure 3(a). (We use the term multicore microarchitecture because the user or programmer is not aware of how the multicore’s components are arranged; it is hidden beneath the multicore ISA layer.) Decomposing the system into replicated subblocks has several benefits:

structures are regularized for scalability and ease of timing closure, systems can be flexibly composed into different topologies, and protocol complexity can be shifted from the component level to the network level. Rather than connecting these tiles with a shared bus, BlackParrot uses a collection of NoCs. The routers used in BlackParrot are dimension-ordered and wormhole-switched, using credit-based flow-control to limit congestion.

In order to promote regularity for hierarchical ASIC flows, the system is designed as a 2-D mesh with a single router per network contained in each tile. Some networks may have multiple endpoints within a tile—these endpoint connections are combined and connected to the router through a wormhole concentrator. While many other processors use standard bus-based interfaces such as AXI or AHB for all on-chip communication, these protocols are highly complex and require sophisticated IP blocks to achieve reasonable performance. Rather than couple its internal networks with any particular implementation of a standard bus, BlackParrot provides a set of adapters to transduce between a set of well-known protocols, such as AXI or WishBone.

BlackParrot Microarchitectural Tile Types

BlackParrot microarchitectural tiles fall into one of the four categories enumerated in Figure 3(a), which we detail below.

BlackParrot Core Tile A BlackParrot Core Tile contains a full BlackParrot processor or an accelerator which acts a processor, comprising one or more coherent caches as well as a directory shard and an L2 slice. A typical system has many Core Tiles.

L2 Extension Tile An L2 extension tile provides a simple scale-out method to increase the amount of on-chip L2 in a BlackParrot system. Each L2 extension contains a directory and a noninclusive nonexclusive L2 slice. By distributing the L2 slices, a system designer can easily change the compute to cache ratio of a BlackParrot system without perturbing critical paths within the cores or the NoCs.

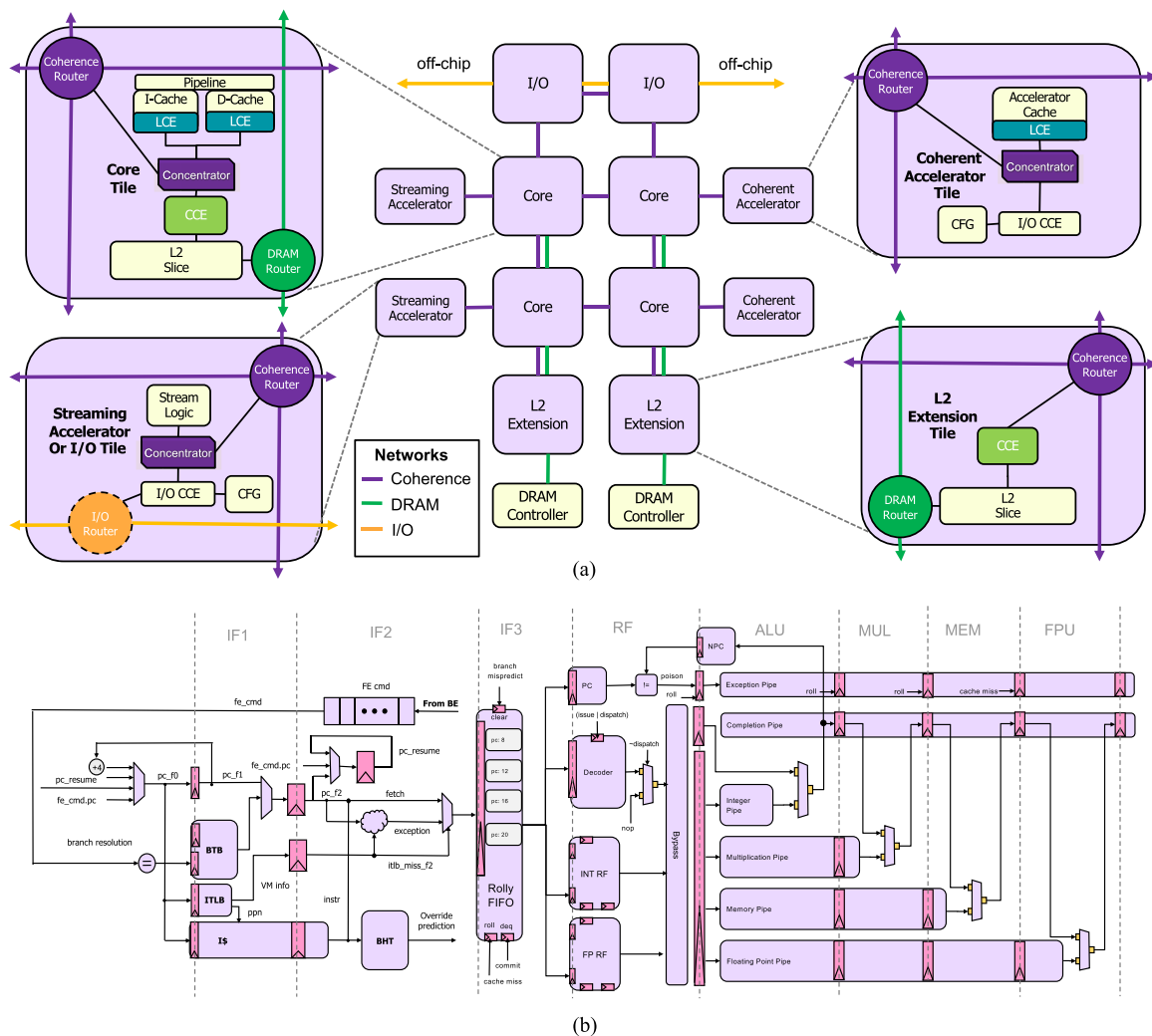


Figure 3. (a) BlackParrot multicore SoCs comprise a mesh of heterogeneous tiles, allowing flexible composition of cores, accelerators, L2 cache slices, I/O, and DRAM controllers. Four different kinds of tiles are pictured. Core tiles implement a processor, a directory shard, and an L2 slice. Coherent Accelerator tiles implement an accelerator that has access to the cache coherent memory system. L2 extension tiles allow the amount of L2 cache to be changed. Streaming accelerator or I/O tiles allow flexible interfacing of a common memory system via a shared non L1-cached address space that is routed over the coherence network. (b) BlackParrot core microarchitecture. Both the front end (IF1 and IF2 stages in the diagram) and back end adhere to the interface specifications, with simple and efficient pipeline implementations. Because of a modest misprediction penalty, complex branch predictors are unnecessary. In order to remove a physical design intensive global stall signal, the back end is nonstalling after the issue stage, instead flushing the pipeline and replaying instructions upon infrequent cache misses. Cache misses are handled entirely through the BedRock coherence system.

Coherent Accelerator Tile Attaching a coherent accelerator tile to the BlackParrot network can be done with a few degrees of specialization. From an abstract system view, a coherent accelerator is simply an LCE with a backing coherent cache. Depending on the

accelerator's needs and the project's complexity budget, users may (in increasing order of complexity):

- 1) attach the accelerator directly to the provided BlackParrot data cache;

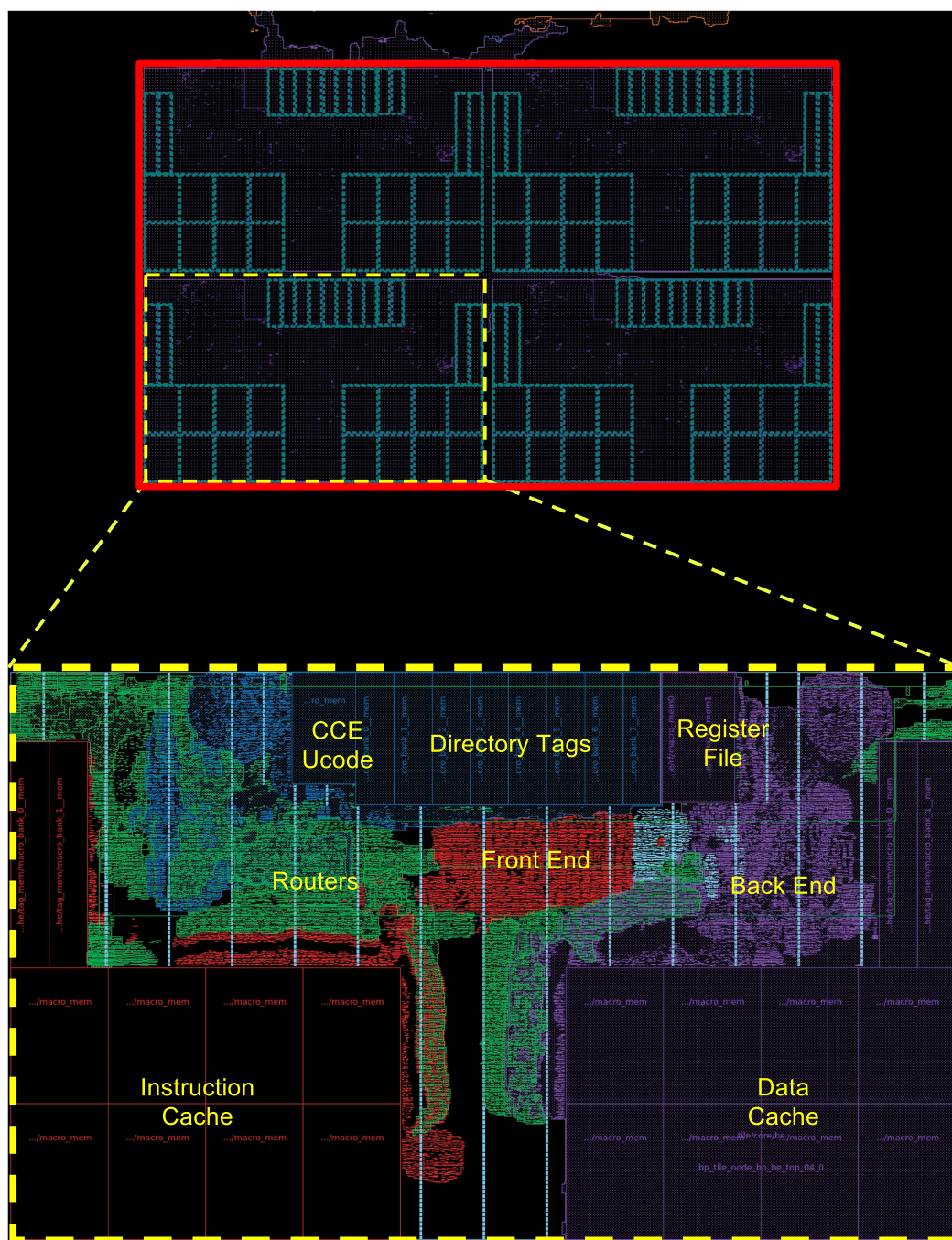


Figure 4. BlackParrot GF12 placed and routed. A quad-core BlackParrot system was taped out in July 2019 using GlobalFoundries 12 nm. Lessons learned from this tapeout have driven several major changes in the BlackParrot system architecture, resulting in a 50% reduction in tile area as well as a 50% reduction in total wire length. Although RTL-level simulation is effective at analyzing and comparing relative performance, an experiment without ASIC validation can mask serious physical limitations of a design.

- 2) reuse the provided BlackParrot LCE and provide a specialized cache;
- 3) provide a specialized LCE implementation that interfaces with directly with the network.

Streaming Accelerator Tile Streaming tiles are tiles which have no locally cached memory and do not logically control any physical memory. That is, these tiles do not contain a backing

coherent cache for its LCE, nor a directory. These tiles may be used for basic I/O devices, network interface links, or even heavyweight streaming-based accelerators such as GPUs.

Other Tile Components

As a truly tiled microarchitecture, BlackParrot distributes as many system resources as possible. Each BlackParrot core tile contains a memory-mapped configuration block, a slice of the core level interrupt controller and a global L2 slice. Splitting these system resources promotes regularity in the tiles, removes globally routed configuration and interrupt signals, ultimately easing physical design implementation. With simple adjustments to the network memory map, components can easily be attached to the BlackParrot system and addressed from any other tile.

Networks-On-Chip

There are 3 NoC classes in BlackParrot: Coherence (BedRock), DRAM, and I/O. Although the NoCs are implemented using standard Base-Jump STL modules, BlackParrot specializes each network for the protocol, including flit width, packet length, and coordinate width, to optimize physical design.

BedRock Network The BedRock network is a cache-coherent fabric connecting all tiles in a BlackParrot system. Specifically, the network is the connection point for all LCEs and CCEs in the system. The BedRock protocol has three logical channels: request, command, and response. The request channel is used by an LCE to initiate a transaction, specifying whether it is a read or write, whether it is cached or uncached, and additional metadata used for return addressing. The command channel is used by a CCE to modify the system's LCE state. Example commands include setting tags, filling data, and completing synchronization sequences. Additionally, LCEs may be commanded to transfer cache lines among each other—these transfers travel over the command network as well. Finally, the response channel is used for coherence acknowledgements, allowing for serialization of requests.

Although the BedRock protocol does not require it, the current implementation of the network is a wormhole-routed 2-D mesh, with one physical channel per logical channel.

DRAM Network The DRAM network connects CCEs to devices which are able to service memory requests, for example DRAM, Flash or on-chip ROMs. Since all requests are initiated by a tile and serviced by memory devices at the bottom of the chip, the memory network is a lightweight 0.5-D network. The DRAM network is particularly suited to wormhole routing, as DRAM controllers tend to return least significant word first.

I/O Network The I/O network exists to connect a BlackParrot processor to peripherals such as serial ports, PCIe controllers, external I/O devices, and debug interfaces. Messages may be initiated on or off chip, so the I/O network is implemented as a 1-D wormhole network. This network only exists in the I/O complex; generally, it serves as a lightweight transducer and physical transport layer between BlackParrot protocols and standard protocols such as AXI, WishBone, and simple bit banging.

Decoupled Core Microarchitecture

BlackParrot is designed to be modular, reaping the usual benefits of simpler verification, more agile development and easier onboarding of users and developers. Additionally, by focusing on interfaces rather than concrete implementations, BlackParrot is provisioned to support a wide variety of possible microarchitectures. Figure 3(b) shows the current BlackParrot core microarchitecture.

Efficiency Through Thin Interfaces

While software interface abstractions are a useful tool, hardware interfaces have physical overheads which can cripple a design. Interfaces in BlackParrot are designed to have minimal overhead, partitioning regions which are both logically and physically separated. Each interface described here is implemented as a parameterizable SystemVerilog struct passed through a latency insensitive port, usually via a small FIFO. Decoupling the Ends ensures there are no timing paths between these logically separated components and provides confidence that implementation changes in one End will not break another.

Front End

The front end presents an in-order but speculative instruction stream to the back end. The issue queue decouples the front end fetch from the back end execution, allowing speculative fetching during long latency back end operations such as servicing cache misses. During instruction fetch, exceptions may arise. Since exceptions in this domain are purely speculative, they are sent to the back end to be serviced in-line with instructions. Because the RISC-V virtual memory scheme may modify architectural state during instruction fetch [setting the “Access (A) bit”], all TLB misses in the front end are sent to the back end to be handled inline with other exceptions. Along with the PC/instruction/exception pair, the front end also sends metadata associated with the branch prediction that resulted in that particular PC fetch.

Back End

The back end executes instructions, handles exceptions, and generally maintains the architectural state of the processor. Messages from the back end to the front end are used to correct mispeculation and update shadow state in the front end. Messages include branch resolution, interrupt redirection, iTLB manipulation, and privilege mode changes. Upon branch resolution, the branch metadata associated with the branch is forwarded back to the front end. This metadata is never inspected by the back end; the particular branch prediction scheme used by the front end is completely opaque to the back end.

Memory End

BlackParrot’s memory end, *BedRock*, is a scalable, distributed, directory-based coherence scheme designed with an emphasis on simplicity and verification. Nodes in the coherence system are either an LCE or CCE. CCEs are responsible for managing coherence for a slice of physical address space. LCEs are responsible for initiating and responding to coherence requests on behalf of a coherent cache. BedRock connects all components of a BlackParrot multicore, including non-coherent or I/O devices.

Three types of CCE are available in Bedrock: a novel microprogrammed variant, a traditional fixed-function management engine, and a

minimal controller which implements only uncached requests, and is used for I/O or simple accelerators. The microprogrammed CCE is the default for a BlackParrot core and provides substantial flexibility and adaptability when implementing variants of coherence protocols, but comes at a small area cost. Notably, since microcode can be changed out by a simple firmware update, advanced coherence experimentation and security patches can be applied even on existing silicon designs.

Agile Development Process

In this section, we describe the key features of the BlackParrot development effort that lead it to be a design users can trust.

Leveraging Open-Source Libraries

In order to rapidly iterate on BlackParrot, it is important to leverage established open-source codebases. By building upon battle-tested hardware libraries such as BaseJump STL for System-Verilog⁸ and Berkeley Verilog HardFloat,⁶ and integrating using latency insensitive design principles, BlackParrot is able to minimize its universe of possible bugs. Development of BlackParrot is done using commercial tools, such as Synopsys VCS, but supporting at least one open-source alternative, such as Verilator,⁷ is a first-class infrastructure concern.

Evaluating Design Complexity and Out-of-Box Experience

One of the most difficult-to-evaluate components of the BlackParrot manifesto is whether we have achieved our goals of virality and complexity. To evaluate this, we assigned BlackParrot as a 3-week class project in an VLSI class attended by 35 fourth and fifth year students. All of the students (including ones that had not taken computer architecture before!) were successful in proposing and implementing unique, previously unsupported features into the core, such as modifying the pipeline to change the load-use latency to two cycles instead of three, adding variable fill widths to the L1 cache, enabling different bank sizes in the L1 cache, and implementing more complex branch predictors in the frontend. We intend to have yearly

BlackParrot-related assignments to continuously self-evaluate these aspects of the project.

Cosimulation Testing Framework

The majority of testing in BlackParrot is done through a system-level testbench driven by program-level execution. In addition to a handful of directed white-box tests, BlackParrot supports the RISC-V tests suite, BEEBS suite, Spec, and CoreMark as a baseline functional regression. Our plan is to greatly expand this selection.

BlackParrot employs a hybrid approach of parallel cosimulation using an open-source RISC-V ISA simulator, Dromajo.¹ First, a long-running program is simulated using Dromajo. Every N cycles, Dromajo collects the architectural state of the system and creates both a memory dump as well as a checkpoint ROM. The checkpoint ROM comprises ordinary RISC-V instructions designed to initialize a freshly rebooted processor to a well-defined architectural state. Next, *in parallel*, the BlackParrot RTL model is restored using each checkpoint ROM and cosimulated alongside the Dromajo model. Imprecise events such as interrupts and device I/O are relayed from RTL to Dromajo to keep the two versions aligned.

ASIC Validation

In addition to our 12-nm BlackParrot chip, which is running in our lab, several upcoming BlackParrot tapeouts are planned both as stand-alone chips and as accelerator hosts. Directories containing tapeout parameterizations, constraints and ASIC infrastructure are all provided as references. Work is in progress to push BlackParrot through the UW OpenROAD Free45 Reference Flow^{2,3} so that users will simply be able to clone the repository and generate a fully placed-and-routed, representative BlackParrot. Providing this capability will enable architects to quickly validate their hardware experiments without the financial and intellectual overheads of maintaining a commercial CAD flow, or needing to sign restrictive foundry NDAs.

CONCLUSION

RISC-V challenges the world order of x86 and ARM. The University of California Berkeley

has bootstrapped a global stewardship to maintain the RISC-V ISA and its software base. However, yet to emerge is a similar global stewardship of a Linux-capable RISC-V implementation that is documented, PPA efficient and implemented in standard SystemVerilog. BlackParrot is architected from the ground up to fill this role, in contrast to prior efforts^{4,5,10} which have cen-

tralized stewardship models and evolved organically to their current state.

BlackParrot is tiny, modular, and friendly. It is an ideal SoC “base class” to integrate with accelerators and build Linux-capable systems with. We welcome your enhancements! As BlackParrot becomes more widely used, community experts can contribute back to BlackParrot, ensuring that it remains representative of state of the art processor designs. BlackParrot is now available on GitHub (<https://github.com/black-parrot/>) under a BSD-3 license.

We welcome your enhancements! As BlackParrot becomes more widely used, community experts can contribute back to BlackParrot, ensuring that it remains representative of state of the art processor designs.

REFERENCES

1. Dromajo. [Online]. Available: <https://github.com/chipsalliance/dromajo/>
2. UW openroad free45 PDK reference flow. [Online]. Available: https://github.com/bsg-idea/uw_openroad_free45/
3. Ajayi *et al.*, “Toward an open-source digital flow: First learnings from the openroad project,” in *Proc. 56th Annu. Design Autom. Conf.*, 2019, Art. no. 76.
4. Asanovic *et al.*, “The rocket chip generator,” UC Berkeley EECS Tech. Rep. UCB/EECS-2016-17.
5. Celio *et al.*, “The Berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized RISC-V processor,” *Electr. Eng. Comput. Sci. Dept.*, Univ. California Berkeley, Tech. Rep. UCB/EECS-2015-167.
6. J. Hauser, *Berkeley HardFloat*. [Online]. Available: <http://www.jhauser.us/arithmetic/HardFloat.html>
7. W. Snyder, “Verilator: Fast, free, but for me?” DVClub Presentation, 2010, p. 11.

8. M. Taylor, "Basejump STL: SystemVerilog needs a standard template library for hardware design," in *Proc. 55th Annu. Design Autom. Conf.*, 2018, Art. no. 73.
9. A. Waterman *et al.*, "The RISC-V instruction set manual, volume i: Base user-level isa," EECS Department, UC Berkeley, Tech. Rep. UCB/EECS-2011-62 116, 2011.
10. F. Zaruba and L. Benini, "The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 11, pp. 2629–2640, Nov. 2019.

Daniel Petrisko is a Ph.D. student at the University of Washington, Seattle. Contact him at petrisko@cs.washington.edu.

Farzam Gilani is a Ph.D. student at the University of Washington, Seattle. Contact him at farzangl@uw.edu.

Mark Wyse is a Ph.D. student at the University of Washington, Seattle. Contact him at wysem@cs.washington.edu.

Dai Cheol Jung is a Ph.D. student at the University of Washington, Seattle. Contact him at dcjung@uw.edu.

Scott Davidson is a Ph.D. student at the University of Washington, Seattle. Contact him at stdavids@uw.edu.

Paul Gao is a Ph.D. student at the University of Washington, Seattle. Contact him at gaozihou@uw.edu.

Chun Zhao is currently a Postdoctoral Research Associate with the University of Washington, Seattle. Contact him at chunzhao@uw.edu.

Zahra Azad is a Ph.D. student at Boston University. Contact her at zazad@bu.edu.

Sadullah Canakci is a Ph.D. student at Boston University. Contact him at scanakci@bu.edu.

Bandhav Veluri is currently working toward the M.S. degree with the University of Washington, Seattle. Contact him at bandhav@uw.edu.

Tavio Guarino is currently working toward the M.S. degree with the University of Washington, Seattle. Contact him at Tavio.guarino@gmail.com.

Ajay Joshi is currently a Professor with Boston University. Contact him at joshi@bu.edu.

Mark Oskin is currently a Professor with the University of Washington, Seattle. Contact him at oskin@cs.washington.edu.

Michael Bedford Taylor is currently a Professor with the University of Washington, Seattle. Contact him at prof.taylor@gmail.com.